ENA Code (C++ version 2.8): User Guide

Building, running and plotting: a guide for users of the code

Contact: Marc Kornbleuth (kmarc@bu.edu)

October 2026

- 1. What the code does
- 2. What you need
  - Software
  - Input files
  - Directory layout
- 3. Building the code
  - On a workstation
  - On Athena
- 4. Settings
  - The settings most users need
  - Energy bands
- 5. Running a steady-state map
  - On a workstation
  - On Athena
- 6. Running time-dependent maps
- 7. The results
- 8. Faster input: converting to HDF5 (optional)
- 9. Making sky maps
- 10. Checking results
- 11. Common problems
- 12. Contact

1. What the code does

The ENA code computes **all-sky maps of energetic neutral atom (ENA) flux** as seen from 1 AU, from a 3D MHD simulation of the heliosphere. For every direction on the sky (a 6° × 6° grid), it sums along the line of sight:

- the ENAs made by charge exchange between heliosheath ions and interstellar hydrogen;
- minus the ENAs lost on their way in.

It reports the result for the energy bands of IBEX-Lo, IBEX-Hi and INCA/Ultra.

It is the C++ successor of the IDL code of Kornbleuth et al. (`make_countrate_files.pro` and its helpers). One program covers the three IDL versions; you choose the model in the settings file:

Model setting (`model.populations`)	**Model**	**IDL original**
`"four_population_ts"`	single-ion, four ion populations from the termination shock (Kornbleuth+2023)	`make_countrate_files.pro`

Model setting (`model.populations`)	Model	IDL original
`"extended_si"`	extended single-ion: four neutral populations, heliosheath-created pickup ions	`make_countrate_files.pro` with `nhpops = 1`
`"multi_ion_inca"`	multi-ion: three ion fluids (Kornbleuth+2026)	`make_countrate_files_inca.pro`

There are two ways to run it:

- **Steady state:** one MHD snapshot gives one set of maps (what the IDL code did).
- **Time-dependent:** a series of snapshots gives maps per time window (e.g. one map per year). This accounts for the travel time of the ENAs and the changing heliosphere along their path.

The maps are plotted with a Python script that places them in ecliptic J2000 coordinates.

Questions about the code: Marc Kornbleuth, kmarc@bu.edu.

2. What you need

Software

	Workstation (Linux or macOS)	NASA HECC Athena
C++ compiler	GCC 9 or newer (with OpenMP)	GCC through the Cray environment (`PrgEnv-gnu`)
Build tool	CMake 3.16 or newer	system CMake
HDF5 library	e.g. `sudo apt install libhdf5-dev`	module `cray-hdf5`
MPI (optional)	e.g. Open MPI, only for time-dependent runs on several processes	`cray-mpich`
Python 3 with numpy, h5py, matplotlib	for the maps and comparison tools	(optional)

Input files

One MHD **snapshot** is four text files, the same files the IDL code read:

File	Contents	Example name
Plasma	density, temperature and velocity of the ion fluid(s) in every grid cell	`opherwithneutralsplasmafinish.dat`
Neutral	interstellar hydrogen (total, plus four populations for the extended single-ion and multi-ion models)	`opherwithneutralsnhfinish.dat`

File	Contents	Example name
Boundaries	the region code of every cell (supersonic wind, heliosheath, beyond the heliopause) and the magnetic field	`boundaries.txt`
Streamlines	the flow streamlines, in Tecplot format	`Streamlines/streamlines_kmhd_moscow_n1200_IB2AU_2AU.dat`

- **Column layouts:** the code knows the column layout of each model's files (single-ion 12 plasma columns, multi-ion 25; the four-population neutral file has 41 columns). Only files written differently need a change in the settings.
- **Grid:** the files must cover the whole grid (by default θ = 0–174°, φ = 0–354° in 6° steps, r = 2–1472 AU in 2 AU steps). A missing or duplicated cell is an error.
- **Voyager 2 trajectory:** `enacpp/configs/v2_trajectory_sim.txt` is used to find the termination-shock distance along Voyager 2. It comes with the code.
- **Time-dependent runs** need one snapshot (four files) per time step, named with a running number, e.g. `plasma_000.dat`, `plasma_001.dat`, ...

Directory layout

Keep this layout; the example settings files find the inputs through it:

```
ENACode/                          the top directory (you run the commands from here)
  enacpp/                         the C++ code (unpacked from enacpp_vX.Y.zip)
    configs/                      settings files
    tools/                        plotting, comparison and job scripts
  secondary_files/input/          the input files (as for the IDL code)
```

3. Building the code

You build once, after unpacking a new version. The commands are run from the top directory `ENACode/`.

On a workstation

1. Configure:

   ```
   cmake -S enacpp -B enacpp/build
   ```

   Read the output: there must be **no** "HDF5 not found" warning.

2. Compile:

   ```
   cmake --build enacpp/build -j
   ```

3. Check the build:

   ```
   enacpp/build/test_m2 enacpp/build/test_tmp
   ```

   Expect `69 checks, 0 failures`.

The program is now `enacpp/build/ena_run`.

On Athena

Athena uses the Cray programming environment, and its login shell is csh. Type each command on its own line.

1. Switch to the GNU compilers and load HDF5:

   ```
   module swap PrgEnv-intel PrgEnv-gnu
   ```

   ```
   module load cray-hdf5
   ```

2. Configure. Use `env CXX=CC` in csh. A steady-state build doesn't need MPI:

   ```
   env CXX=CC cmake -S enacpp -B enacpp/build_nompi -DENA_WITH_MPI=OFF -DHDF5_ROOT=$HDF5_
   DIR -DCMAKE_CXX_FLAGS=-I$HDF5_DIR/include
   ```

   For time-dependent runs on several MPI processes, make a second build with MPI: the same line with `enacpp/build_mpi` and without `-DENA_WITH_MPI=OFF`. The MPI build has not yet been tried on Athena; check that the `cmake` output reports MPI as found.

3. Compile:

   ```
   cmake --build enacpp/build_nompi -j 8
   ```

4. Check:

   ```
   enacpp/build_nompi/test_m2 enacpp/build_nompi/test_tmp
   ```

   Expect `69 checks, 0 failures`.

The messages "No supported cpu target is set, CRAY_CPU_TARGET=x86-64 will be used" are harmless. To hide them, first run `setenv CRAY_CPU_TARGET x86-64`.

4. Settings

All settings are in one text file in TOML format: `name = value` lines grouped under `[section]` headers, with `#` starting a comment.

`enacpp/configs/all_settings.toml` is the master file. It lists every setting with its default and explanation, like `load_parameters.pro` did for the IDL code. To make your own:

1. Copy it:

   ```
   cp enacpp/configs/all_settings.toml enacpp/configs/my_run.toml
   ```

2. Edit the copy.

3. Run the copy.

Shorter examples: `run_reference.toml` (single-ion), `run_esi.toml` (extended single-ion), `run_mi.toml` (multi-ion), `run_esi_td.toml` (time-dependent).

Paths:

- **Input directory (`inputs.dir`):** relative to the settings file.
- **Output directory (`run.output_dir`):** relative to the directory you run from.

The settings most users need

Setting	What it does	Default
`[model] populations`	which model (table in section 1)	`"four_population_ts"`
`[model] source`	`"standard"` (BU MHD model) or `"moscow"` (Moscow model, single-ion only)	`"standard"`
`[bands] run`	the energy bands to compute, e.g. `[3, 6, 9, 13, 14]` (table below)	`[13]`
`[run] output_dir`	where the results go	`"output"`
`[inputs] dir`	the directory of the input files	`"."`
`[inputs] format`	`"ascii"` (text files) or `"hdf5"` (converted files, see section 8)	`"ascii"`
`[[snapshot]]`	the four input file names of a steady-state run	
`[integration] r_lower`, `r_upper`	the radial range of the line-of-sight sum, AU	2, 800
`[physics] survival`	ENA losses on the way in	`true`
`[physics] extinction`	cooling of the pickup ions along the streamlines	`true`
`[physics] kappa`, `kappa_pui`, `kappa_ref`, `kappa_oh`	kappa values of the ion distributions (≤ 1.5 means Maxwellian)	1
`[time_dependent] enabled`	`true` for time-dependent maps (section 6)	`false`

The remaining settings are expert options, documented in `all_settings.toml`: the termination-shock treatment, `[region6]`, `[multi_ion]`, sensor tables and input column layouts. Leave them at their defaults unless you know you need them.

Changing one setting for one run. Any setting can be overridden on the command line, without editing the file:

```
enacpp/build/ena_run enacpp/configs/my_run.toml --set physics.kappa=1.6 --out kappa16
```

Energy bands

Extended single-ion and multi-ion models. The single-ion model uses the IBEX-Lo energies of `load_parameters.pro` and has INCA bands 14–17 only.

Bands	Instrument	Central energies (keV)
1–7	IBEX-Lo	0.048 (band 1), 0.082, 0.147, 0.26, 0.51, 0.975, 1.97 (band 7); band 0 has no central energy in this table

Bands	Instrument	Central energies (keV)
9–13	IBEX-Hi	0.71, 1.11, 1.74, 2.73, 4.29 (band 8 is not usable)
14–20	INCA / Ultra	8.38, 18.0, 29.0, 43.9, 33.5, 70, 80

5. Running a steady-state map

On a workstation

From the top directory:

```
enacpp/build/ena_run enacpp/configs/my_run.toml
```

Useful options:

Option	Effect
`--bands 3,6,9,13,14`	run these bands (overrides `[bands] run`)
`--out DIR`	write the results to DIR (overrides `output_dir`)
`--threads N`	use N processor cores (default: all)
`--set section.name=value`	override one setting (repeatable)
`--no-legacy`	do not write the IDL-format `.dat` files

The program prints its settings, checks of the inputs, and its progress. A band takes seconds to a few minutes, depending on the machine and the number of cores. The results are the same for any number of cores.

On Athena

Runs go through the PBS batch system. `enacpp/tools/run_athena_steady.pbs` is a ready job script for one whole node (256 cores).

1. **Check the settings** at the top of the script:
   - the queue (`normal`);
   - your group (`#PBS -W group_list=…`);
   - the settings file (`CONFIG=`);
   - the bands (`BANDS=`).

   PBS does not allow a comment after a `#PBS` line on the same line.

2. **Submit** from the top directory:

   ```
   qsub enacpp/tools/run_athena_steady.pbs
   ```

3. **Check the status:**

   ```
   qstat -u $USER
   ```

   `Q` means queued, `R` running.

4. **Follow the log** while it runs:

```
tail -f ena_steady_<jobnumber>.log
```

Ctrl-C stops the display, not the job.

The script runs the map twice, with 256 and with 16 threads, as a check. For production runs set `CHECK_THREADS=0` to skip the second run.

6. Running time-dependent maps

1. **Name the snapshots with a running number.** Each MHD snapshot is four files, e.g. `time_series/plasma_000.dat`, `plasma_001.dat`, ...
2. **Set the `[time_dependent]` section** in your settings file:

   ```
   [time_dependent]
   enabled = true
   start = 2005.0          # time of the first snapshot (years)
   end = 2025.0            # end of the last snapshot's time step
   dt_months = 2           # snapshot spacing
   output_start = 2009.0   # first output window
   output_end = 2025.0
   cadence = 1.0           # one map per year

   [time_dependent.files]
   first_index = 0
   plasma = "time_series/plasma_{index:03}.dat"
   neutral = "time_series/neutral_{index:03}.dat"
   boundaries = "time_series/boundaries_{index:03}.txt"
   streamlines = "time_series/streamlines_{index:03}.dat"
   ```

   (end − start)/dt snapshots must exist. Each snapshot holds unchanged for its time step; there is no interpolation between snapshots.
3. **Run** as in section 5. With an MPI build, the run can be spread over several processes, e.g. 8 processes of 16 threads each:

   ```
   mpiexec -n 8 enacpp/build_mpi/ena_run enacpp/configs/my_td_run.toml --threads 16
   ```

   `enacpp/tools/run_athena.pbs` is a job-script template for this.

Regions with missing flux. ENAs from far away take years to arrive. So in the first windows, some lines of sight are still missing ENAs that were made before the first snapshot. These lines of sight are:

- set to 0 by default (`include_incomplete_lines = true` keeps their partial flux);
- flagged in the output, and drawn with a box on the maps;
- listed with a WARNING in the log and in `time_dependent_summary.txt`.

Start the input series several years before the first window you want. Low energies need the longest: about 10 years at 0.15 keV.

7. The results

In the output directory:

File	Contents
`flux_bandNN.h5`	the flux maps of band NN (cm^{-2} s^{-1} sr^{-1} keV^{-1}), one per component (e.g. `transmitted`, `reflected`, `accelerated`, `solar_wind`, `total`), plus the settings and code version used
`flux_bandNN_<window>.h5`	the same for a time window (time-dependent runs), plus the coverage flags
`Opher_multiion_withneutrals<TAG>CountRateKappa<k>EbandNN.dat`	count rates in the old IDL format (tags PUI1, PUI2, PUI3, SW, CombPUIs, ...), for existing IDL scripts
`time_dependent_summary.txt`	one line per band and window, with the coverage (time-dependent runs)
`heliosheath_pockets.txt`	lines of sight that cross the termination shock more than once (if any)

Maps are arrays of 30 × 60 values: θ = 0, 6, ..., 174° (north pole first) by φ = 0, 6, ..., 354°, in the simulation frame.

8. Faster input: converting to HDF5 (optional)

Reading the text files takes most of the time of a run. For many runs on the same snapshots, especially time-dependent runs, convert them once to binary HDF5 files:

```
enacpp/build/ena_convert enacpp/configs/my_run.toml snapshots_h5
```

The program prints the lines to put in the settings file: `format = "hdf5"`, and the new file names. The maps are identical to those from the text files.

9. Making sky maps

`enacpp/tools/ena_plot.py` draws the maps in ecliptic J2000 coordinates, in a Mollweide projection (Python 3 with numpy, h5py and matplotlib):

```
python3 enacpp/tools/ena_plot.py OUT/flux_band13.h5
python3 enacpp/tools/ena_plot.py OUT/flux_band13.h5 --view tail
python3 enacpp/tools/ena_plot.py OUT/flux_band*.h5 --config enacpp/configs/plot_settings.tom
l --out maps
```

Option	Effect
`--view nose` / `tail` / `ecliptic`	centre on upwind (default), downwind, or ecliptic (0°, 0°)
`--scale S`	multiply the flux by S; the title shows "(Flux x S)"
`--vmin A --vmax B`	colour-bar range (default: per-band limits, else the map's own range)
`--legend-style integer` / `float` / `exponent`	colour-bar number format

Option	Effect
`--component accelerated`	plot one component (default `total`)
`--log`	logarithmic colour scale
`--format pdf`	PDF (or png, eps, svg)
`--idl --projection hammer`	the placement and projection of the old IDL maps, for comparison

- **Settings file:** `enacpp/configs/plot_settings.toml` holds all plot settings, like the plotting part of `load_parameters.pro`.
- **Old IDL output:** the plotter also reads the IDL count-rate files (`…CountRate…EbandNN.dat`).

10. Checking results

Tool	Use
`python3 enacpp/tools/same_maps.py DIR1 DIR2`	are two runs bit-identical (same machine)?
`python3 enacpp/tools/compare_runs.py REF OTHER`	do two runs from different machines agree (e.g. workstation and Athena)? Reports each map as identical, agreeing within 1e-9, or different, with the worst cells.

11. Common problems

Problem	Solution
`cmake` reports “HDF5 not found”, or compiling stops at `hdf5.h`	Load HDF5 first (`module load cray-hdf5` on Athena; `libhdf5-dev` on Ubuntu), delete the build directory, and configure again. On Athena add `-DHDF5_ROOT=$HDF5_DIR -DCMAKE_CXX_FLAGS=-I$HDF5_DIR/include`.
`CXX=CC: Command not found`	Your shell is csh: write `env CXX=CC cmake …`.
`qsub: directive error`	A `#PBS` line has a comment after it; remove the comment.
“cannot open” an input file	`inputs.dir` is relative to the settings file, not to where you run from.
A run on many cores is no faster than on a few	The threads are all on one core: in a job script, set `OMP_PROC_BIND=spread` and `OMP_PLACES=cores` (as `run_athena_steady.pbs` does), and do not start a steady-state run with `mpiexec`.
WARNING about regions with missing flux	Time-dependent run: the first windows are too close to the first snapshot for that energy (section 6).
WARNING about region-5 cells or heliosheath pockets	The region codes in `boundaries.txt` have undetermined cells or a wavy termination shock; the log and `heliosheath_pockets.txt` give their locations.

More detail:

- `enacpp/README.md`: the full reference.
- `enacpp/CHANGES_FROM_IDL.md`: every difference from the IDL code.

12. Contact

For questions about the code, its inputs or its results, contact **Marc Kornbleuth**, kmarc@bu.edu.